

[Completar institucion]

[Completar carrera]

Diseño e implementación de un sniffer ARINC 429 lógico con captura pasiva, transporte SPI y visualización web

Trabajo Final de Grado

Autor: Matias Diaz Benini
Tutor / Director: [Completar]
Fecha: Junio de 2026

Proyecto PAMPA / ARINC 429

Dedicatoria

A quienes acompañaron el desarrollo del trabajo, sostuvieron las pruebas de laboratorio y aportaron criterio técnico para convertir una idea experimental en un sistema medible, repetible y documentado.

Agradecimientos

Se agradece al tutor del proyecto por las observaciones técnicas realizadas durante el desarrollo, especialmente las referidas a integridad de datos, transporte por tramas completas, errores operativos en cero, mediciones de osciloscopio y robustez ante fallas. También se agradece la disponibilidad de instrumental, placas y espacio de laboratorio que permitieron validar el sistema durante corridas prolongadas.

Hoja de aceptacion

Titulo del trabajo:	Diseno e implementacion de un sniffer ARINC 429 logico con captura pasiva, transporte SPI y visualizacion web
Autor:	Matias Diaz Benini
Tutor / Director:	[Completar]
Institucion:	[Completar institucion]
Carrera:	[Completar carrera]
Fecha:	Junio de 2026

Firma del tutor

Firma del evaluador

Firma del evaluador

Firma del evaluador

Índice general

Dedicatoria	I
Agradecimientos	II
Hoja de aceptacion	III
Glosario y convenciones	IX
Resumen	X
Palabras claves	1
1. Introduccion	2
1.1. Problema abordado	2
1.2. Alcance	3
2. Objetivos	4
2.1. Objetivo general	4
2.2. Objetivos especificos	4
2.3. Destinatarios	4
2.4. Beneficios	5
3. Estudio tecnico	6
3.1. Modelo ARINC 429 utilizado	6
3.2. Formato de palabra	6
3.3. Labels utilizados	7
3.4. Codificacion bipolar logica con retorno a cero	7
3.5. Seleccion tecnologica	7
3.6. Alternativas analizadas	8
3.6.1. UART, I2C y SPI	8
3.6.2. SPI byte a byte y SPI por trama	8
3.6.3. Polling y DRDY	8
3.6.4. Wi-Fi y Ethernet directo	8
4. Arquitectura del sistema	9
4.1. Vista general	9
4.2. Distribucion fisica	10
4.3. Red y servicios	10

5. Desarrollo del hardware logico	11
5.1. ARINC-TX	11
5.2. ARINC-RX	11
5.3. ARINC-SNIFFER	12
5.3.1. Filtro e integridad	12
5.3.2. Deteccion automatica de velocidad	13
5.3.3. Recuperacion de captura	13
6. Transporte SPI y software host	14
6.1. Protocolo SPI	14
6.2. SPI por PIO-frame	14
6.3. DRDY	15
6.4. Bridge HTTP/SPI	15
6.5. Dashboard Flask	16
6.6. Prometheus y Grafana	16
6.7. Supervisor	16
7. Metodologia de desarrollo y validacion	17
7.1. Procedimiento general	17
7.2. Actividades realizadas	17
7.3. Instrumental y medios	18
8. Resultados	19
8.1. Validacion con osciloscopio	19
8.2. Barrido de frecuencia SPI	20
8.3. Prueba de paridad estricta	21
8.4. Corridas finales autorate	21
8.5. Balance de contadores	22
8.6. Estado del supervisor	22
9. Costos, operacion e impacto	24
9.1. Inversion requerida	24
9.2. Costos de operacion y mantenimiento	24
9.3. Viabilidad comercial	25
9.4. Impacto ambiental	25
9.5. Impacto social y academico	25
10.Dificultades y soluciones aplicadas	26
10.1. Errores SPI de arranque	26
10.2. Transporte byte a byte	26
10.3. Polling	26
10.4. Reinicio o desconexion del sniffer	26
10.5. Gap temporal entre bits	26
10.6. Robustez de servicios	27
11.Conclusiones	28
12.Referencias y bibliografia	29

A. Comandos operativos principales	31
A.1. Servicios en Raspberry Pi 3B+	31
A.2. Prometheus y Grafana en notebook	31
A.3. Registro estadístico	31
B. Archivos de evidencia	32

Índice de figuras

3.1. Representacion conceptual de la codificacion bipolar logica con retorno a cero.	7
4.1. Arquitectura general del sistema implementado. El sniffer se conecta al cable observado, no a una placa como emisor.	9
4.2. Servicios en host y observabilidad. El bridge no se expone a la red externa.	10
5.1. Flujo interno del transmisor.	11
5.2. Flujo interno del receptor.	12
5.3. Flujo interno del sniffer.	12
8.1. Captura MATH de senal diferencial logica. Se observan pulsos positivos, negativos y nivel nulo.	19
8.2. Secuencia de palabras sobre el canal observado con representacion diferencial.	20
8.3. Medicion de canales logicos individuales. Los canales no se pisan: represen- tan polaridades opuestas o estado nulo.	20

Índice de tablas

3.1. Campos principales de la palabra ARINC implementada	6
3.2. Labels utiles de la plataforma	7
3.3. Tecnologias seleccionadas	7
4.1. Placas y responsabilidades	10
6.1. Estructura del paquete SPI	14
6.2. Comandos SPI utilizados	14
7.1. Actividades principales del trabajo	17
8.1. Resumen del barrido SPI	21
8.2. Prueba de paridad estricta	21
8.3. Comparacion de corridas finales	22
9.1. Rubros de inversion	24

Glosario y convenciones

Termino	Descripcion
ARINC 429	Bus aeronautico simplex basado en palabras de 32 bits y senalizacion bipolar con retorno a cero.
FWD	Canal directo observado en el laboratorio, desde la placa transmisora hacia la receptora.
REV	Canal inverso disponible como segundo canal observable por el sniffer.
RZ	Return to Zero. Cada bit vuelve a nivel nulo durante parte de su celda temporal.
Label	Campo de 8 bits que identifica el significado de una palabra ARINC.
SDI	Source/Destination Identifier. Campo usado como subidentificador de fuente o destino.
SSM	Sign/Status Matrix. Campo usado para estado o signo de la palabra.
PIO	Programmable I/O de Raspberry Pi Pico, usado para temporizacion deterministica.
SPI	Serial Peripheral Interface, usado entre el sniffer y la Raspberry Pi 3B+.
DRDY	Data Ready. Linea digital que indica a la 3B+ que hay un snapshot disponible.
Bridge	Servicio Python en la 3B+ que traduce SPI a HTTP/JSON y metricas.
Supervisor	Servicio de vigilancia que controla bridge y dashboard, y clasifica el estado del sistema.

Resumen

El trabajo desarrolla una plataforma de laboratorio para generar, recibir, capturar y visualizar trafico compatible a nivel logico y temporal con ARINC 429. La solucion implementa un transmisor, un receptor, un sniffer pasivo, un bridge SPI/HTTP en Raspberry Pi 3B+ y un dashboard web con metricas para Prometheus y Grafana. El transmisor genera palabras de 32 bits con codificacion bipolar logica con retorno a cero, operando a 100 kbps o 12.5 kbps. El sniffer observa el canal sin intervenirlo, filtra por label y SDI, verifica paridad, detecta automaticamente la velocidad del enlace y entrega snapshots al host por SPI en tramas completas de 32 bytes. La Raspberry Pi 3B+ publica los datos por HTTP y supervisa los servicios mediante un proceso independiente. La validacion incluyo mediciones de osciloscopio, prueba de paridad estricta, barrido de frecuencia SPI, recuperacion ante fallas y dos corridas finales de ocho horas. Los ensayos finales cerraron con cero errores SPI operativos, cero errores de paridad, cero overflow, cero drops SPI y deteccion estable de 100 kbps y 12.5 kbps. El resultado es un sistema reproducible, observable y documentado para analisis de trafico ARINC 429 en condiciones controladas de laboratorio.

Palabras claves

ARINC 429; Raspberry Pi Pico; PIO; SPI; sniffer pasivo; Flask; Prometheus; Grafana; retorno a cero; avionica; adquisicion de datos.

Capítulo 1

Introduccion

ARINC 429 es una arquitectura de comunicacion ampliamente difundida en sistemas aeronauticos. Su interes tecnico surge de combinar una palabra de datos fija, un canal simplex, una temporizacion estricta y una codificacion fisica con retorno a cero. Estas características permiten estudiar problemas de temporizacion, integridad, captura pasiva y observabilidad de datos en tiempo real.

El presente trabajo implementa una plataforma experimental que reproduce el comportamiento logico y temporal de un enlace ARINC 429 mediante placas Raspberry Pi Pico y una Raspberry Pi 3B+. El sistema no se limita a transmitir datos: incorpora un sniffer pasivo, un transporte robusto hacia host, un dashboard operativo, metricas historicas y procedimientos de validacion. La motivacion principal es disponer de una herramienta controlada y repetible para analizar trafico ARINC 429, verificar palabras, medir temporizacion y demostrar robustez de captura durante corridas prolongadas.

El alcance del trabajo se define sobre senales logicas de 3.3 V generadas en laboratorio. Dentro de ese alcance se implemento la palabra ARINC, la codificacion bipolar logica con retorno a cero, la transmision a 100 kbps y 12.5 kbps, la captura pasiva, la deteccion automatica de velocidad, el rechazo por paridad, el transporte SPI por tramas completas, la visualizacion web y la observabilidad por Prometheus/Grafana.

1.1. Problema abordado

La captura de un enlace tipo ARINC exige resolver simultaneamente varios aspectos:

- generar una temporizacion estable e independiente de la carga de CPU;
- reconstruir palabras de 32 bits a partir de dos lineas logicas;
- capturar el enlace sin modificar la comunicacion observada;
- transportar datos hacia un host sin perder integridad;
- distinguir errores de arranque de errores operativos reales;
- registrar evidencias objetivas de funcionamiento.

El sistema construido resuelve estos puntos mediante PIO en las Pico, transporte SPI por trama completa, linea DRDY, supervisor de servicios y registro estadistico automatizado.

1.2. Alcance

El alcance tecnico validado incluye:

- transmision FWD sobre dos GPIO en modo bipolar logico;
- recepcion de palabras FWD por una segunda Pico;
- sniffer pasivo sobre FWD y REV;
- labels de telemetria: temperatura, velocidad y altitud;
- filtro por label y SDI;
- paridad impar de palabra ARINC;
- deteccion autonoma de 100 kbps y 12.5 kbps;
- enlace SPI a 8 MHz entre sniffer y 3B+;
- dashboard Flask y metricas Prometheus;
- corridas finales de ocho horas.

Capítulo 2

Objetivos

2.1. Objetivo general

Disenar, implementar y validar una plataforma de laboratorio capaz de generar, recibir, capturar y visualizar trafico ARINC 429 logico, con sniffer pasivo, transporte SPI robusto y observabilidad web.

2.2. Objetivos especificos

1. Implementar un transmisor capaz de emitir palabras ARINC de 32 bits con retorno a cero a 100 kbps y 12.5 kbps.
2. Implementar un receptor que reconstruya palabras a partir del par logico FWD y verifique integridad.
3. Implementar un sniffer pasivo que observe FWD y REV sin transmitir sobre el enlace ARINC.
4. Filtrar palabras por label y SDI, y rechazar palabras con paridad incorrecta.
5. Transportar snapshots y estadisticas hacia una Raspberry Pi 3B+ por SPI en tramas completas.
6. Publicar datos operativos mediante HTTP/JSON, dashboard Flask y metricas Prometheus.
7. Registrar evidencias en CSV, JSON y PDF.
8. Validar estabilidad mediante osciloscopio y corridas prolongadas.

2.3. Destinatarios

El trabajo esta destinado a docentes, evaluadores y estudiantes de electronica, telecomunicaciones, sistemas embebidos y avionica que requieran una plataforma practica para estudiar comunicacion ARINC 429 a nivel logico y temporal.

2.4. Beneficios

Los beneficios principales son:

- plataforma reproducible para ensayos de comunicacion avionica;
- separacion clara entre generacion, recepcion, sniffing y visualizacion;
- instrumentacion directa con osciloscopio;
- trazabilidad de resultados por JSON, CSV y PDF;
- comparacion entre 100 kbps y 12.5 kbps;
- validacion de integridad con errores operativos en cero;
- bajo costo de implementacion al usar placas de desarrollo disponibles.

Capítulo 3

Estudio tecnico

3.1. Modelo ARINC 429 utilizado

ARINC 429 define una comunicacion simplex: un transmisor emite palabras hacia uno o mas receptores. En el modelo implementado se trabaja con dos lineas logicas por canal, denominadas A y B. Para representar la senal bipolar:

- bit logico alto: A bajo, B alto en la representacion de la PIO usada;
- bit logico bajo: A alto, B bajo;
- nivel nulo: A bajo y B bajo.

La observacion diferencial mediante osciloscopio, en modo MATH, permite ver valores aproximados de +3.3 V, 0 V y -3.3 V. En el laboratorio esta amplitud es logica y no corresponde a una interfaz electrica certificada de campo.

3.2. Formato de palabra

Cada palabra transporta 32 bits. La implementacion organiza la informacion en label, SDI, dato, SSM y paridad impar.

Tabla 3.1: Campos principales de la palabra ARINC implementada

Campo	Bits	Uso en el sistema
Label	1–8	Identificador de variable
SDI	9–10	Subidentificador de fuente/destino
Dato	11–29	Magnitud codificada
SSM	30–31	Estado de la palabra
Paridad	32	Paridad impar

3.3. Labels utilizados

Tabla 3.2: Labels utiles de la plataforma

Label	SDI	Nombre	Unidad
0xA5	0	TEMPERATURA	C
0xB1	1	VELOCIDAD	kt
0xC2	2	ALTITUD	ft
0xAC	3	ACK_BATCH	batch

La operacion final utiliza principalmente temperatura, velocidad y altitud. El label ACK_BATCH queda contemplado en el protocolo y en el filtro para observar trafico inverso cuando exista actividad por REV.

3.4. Codificacion bipolar logica con retorno a cero

Cada bit se divide en dos mitades:

- primera mitad: nivel activo, positivo o negativo;
- segunda mitad: retorno a nivel nulo.

A 100 kbps el bit dura 10 us: 5 us activos y 5 us nulos. A 12.5 kbps el bit dura 80 us: 40 us activos y 40 us nulos.

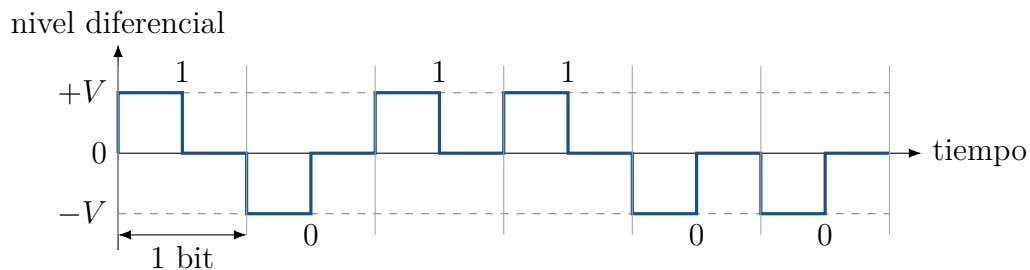


Figura 3.1: Representacion conceptual de la codificacion bipolar logica con retorno a cero.

3.5. Seleccion tecnologica

La seleccion se baso en disponibilidad, temporizacion deterministica y capacidad de observacion.

Tabla 3.3: Tecnologias seleccionadas

Elemento	Tecnologia	Fundamento
Transmision y recepcion	Raspberry Pi Pico / PIO	Permite temporizacion por hardware sin depender del lazo principal de CPU.

Elemento	Tecnologia	Fundamento
Sniffer	Raspberry Pi Pico / PIO	Permite observar dos canales y ejecutar SPI slave por trama completa.
Host	Raspberry Pi 3B+	Dispone de Linux, SPI, Ethernet, Python y servicios HTTP.
Visualizacion	Flask	Dashboard liviano para operacion directa.
Historico	Prometheus/Grafana	Scraping y visualizacion temporal de metricas.
Transporte sniffer-host	SPI a 8 MHz	Margen suficiente para snapshots sin limitar el enlace ARINC.
Arranque de servicios	systemd	Autoarranque, reinicio automatico y trazabilidad por logs.

3.6. Alternativas analizadas

3.6.1. UART, I2C y SPI

UART fue descartado como transporte principal entre sniffer y host por su menor capacidad para transacciones estructuradas con reloj controlado por el host. I2C ofrece direccionamiento simple, pero no resulta conveniente para rafagas de telemetria y snapshots a alta tasa. SPI permite que la Raspberry Pi 3B+ actue como master, controle el reloj y lea estructuras completas de 32 bytes con bajo overhead.

3.6.2. SPI byte a byte y SPI por trama

El modo byte a byte fue util como base estable. La implementacion final usa tramas completas de 32 bytes en PIO porque responde mejor al requisito de transportar paquetes atomicos, reduce ambigüedades de sincronizacion y permite un protocolo mas auditable.

3.6.3. Polling y DRDY

El polling periodico funciona, pero realiza consultas aun cuando no hay datos nuevos. El modo DRDY usa una linea adicional: el sniffer sube GP20 cuando hay snapshot nuevo y la 3B+ consulta por SPI. El sistema conserva timeout como respaldo si se pierde una transicion.

3.6.4. Wi-Fi y Ethernet directo

La configuracion recomendada usa Ethernet directo entre notebook y 3B+. Esta red reduce jitter operativo y mantiene separada la comunicacion local del trafico Wi-Fi.

Capítulo 4

Arquitectura del sistema

4.1. Vista general

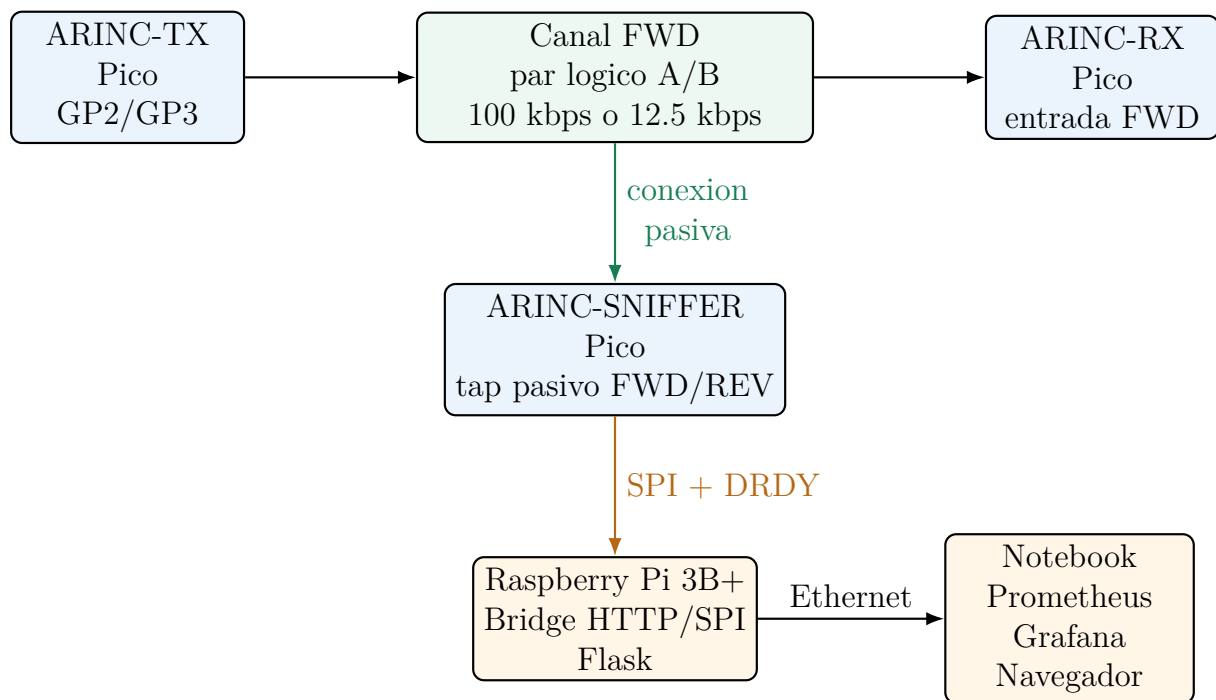


Figura 4.1: Arquitectura general del sistema implementado. El sniffer se conecta al cable observado, no a una placa como emisor.

4.2. Distribucion fisica

Tabla 4.1: Placas y responsabilidades

Placa	Funcion
ARINC-TX	Genera palabras utiles y ruido controlado, selecciona velocidad y transmite FWD.
ARINC-RX	Recibe FWD, reconstruye palabras, valida paridad y procesa stream.
ARINC-SNIFFER	Observa FWD y REV, filtra, valida paridad, mantiene snapshots y responde por SPI.
Raspberry Pi 3B+	Ejecuta bridge, dashboard Flask, recorder y supervisor.
Notebook	Visualiza dashboard, Prometheus y Grafana por Ethernet.

4.3. Red y servicios

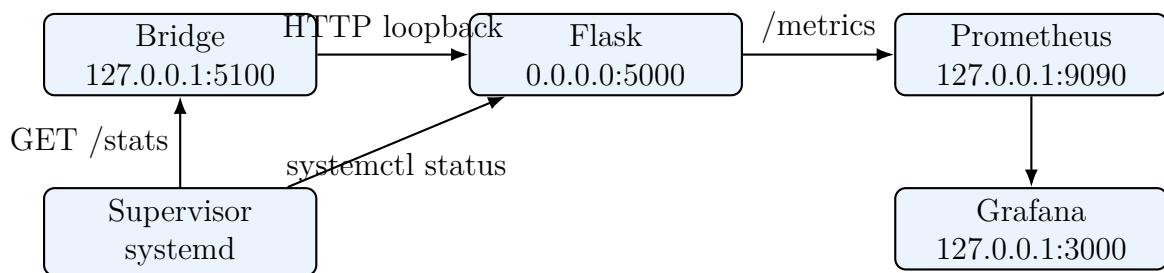


Figura 4.2: Servicios en host y observabilidad. El bridge no se expone a la red externa.

Capítulo 5

Desarrollo del hardware logico

5.1. ARINC-TX

La placa transmisora genera una secuencia de palabras de telemetria. El target principal es:

```
arinc_tx_arinc429_logic_stream_aurorate
```

Este firmware selecciona al arrancar una de dos velocidades:

- 100000 bps;
- 12500 bps.

La seleccion se realiza en firmware mediante una semilla basada en tiempo de arranque. Para ensayos controlados existe tambien el target `arinc_tx_arinc429_logic_stream_12k5`, que fuerza 12.5 kbps.

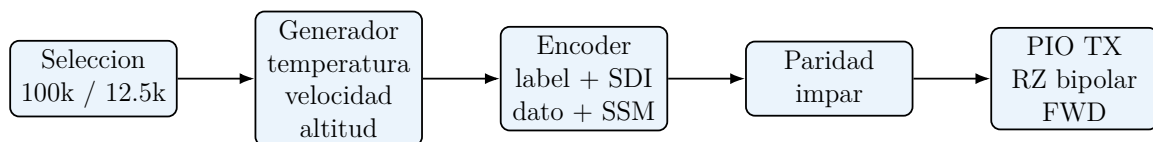


Figura 5.1: Flujo interno del transmisor.

La PIO recibe la palabra por FIFO, desplaza 32 bits y genera dos salidas. Para cada bit emite una mitad activa y luego un nivel nulo. La temporizacion se define por divisor de clock, por lo que el tiempo de bit no depende del tiempo de ejecucion del lazo principal.

5.2. ARINC-RX

La placa receptora observa el par FWD. Su state machine PIO espera actividad en cualquiera de los hilos, clasifica el simbolo segun el pin activo, espera el retorno a nulo y reconstruye 32 bits. El target final es:

```
arinc_rx_arinc429_logic_stream
```

El receptor no necesita que el transmisor le informe la velocidad. El criterio de recepcion se basa en actividad y retorno a nulo, no en una espera fija de 10 us. Por eso puede reconstruir palabras tanto a 100 kbps como a 12.5 kbps, siempre que la forma RZ sea respetada.



Figura 5.2: Flujo interno del receptor.

5.3. ARINC-SNIFFER

El sniffer es el nucleo del trabajo. Observa FWD y REV sin transmitir sobre esas lineas. La comunicacion activa del sniffer ocurre exclusivamente hacia la Raspberry Pi 3B+ por SPI.

El target usado es:

```
sniffer_arinc429_logic_pio_frame
```

La build validada informa:

```
SPI-PIOFRAME-ARINC429-STRICTPARITY-DRDY-AUTORATE-RECOVERY-V5
```

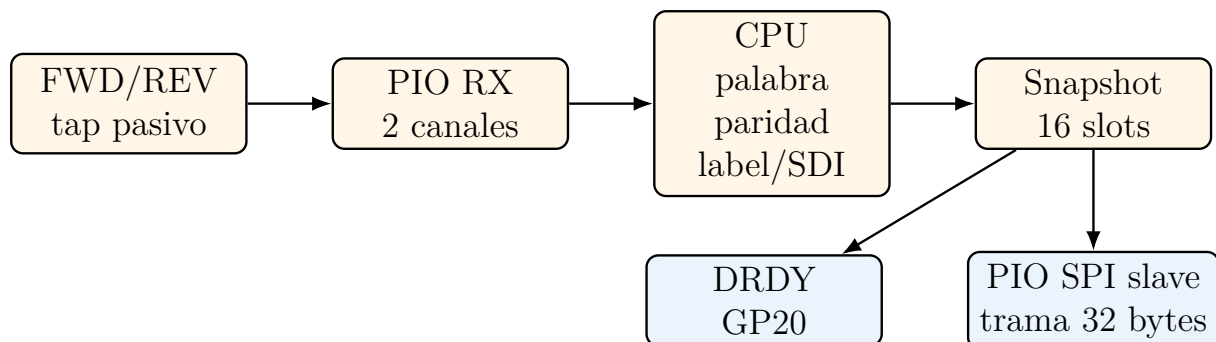


Figura 5.3: Flujo interno del sniffer.

5.3.1. Filtro e integridad

El sniffer usa una whitelist de hasta 10 entradas. En la configuracion de validacion:

```
0xA5/0
0xB1/1
0xC2/2
0xAC/3
```

Una palabra se acepta si:

1. coincide con label y SDI permitidos;
2. cumple la paridad impar;
3. su magnitud es plausible para la variable decodificada;

4. corresponde al canal esperado cuando se trata de ACK.

Las palabras con paridad incorrecta incrementan el contador de error, pero no actualizan el snapshot ni `accepted_words`.

5.3.2. Deteccion automatica de velocidad

El sniffer calcula una tasa de palabras FWD dentro de una ventana de 2 s. Si la tasa supera el umbral alto, clasifica el enlace como 100 kbps. Si supera el umbral bajo pero no el alto, lo clasifica como 12.5 kbps. El resultado se publica como:

<code>detected_bit_rate_bps</code> <code>detected_bit_rate_txt</code>
--

En SPI se transporta en el payload de meta como `detected_bit_rate_hz_div100`. La 3B+ lo multiplica por 100 para reconstruir la velocidad en bps.

5.3.3. Recuperacion de captura

Ante una resincronizacion, la state machine se detiene, limpia FIFO, reinicia su estado interno y espera un intervalo de canal nulo antes de volver a capturar. El tiempo de reposo requerido cambia segun la velocidad detectada. Esto evita reengancharse en medio de una palabra.

Capítulo 6

Transporte SPI y software host

6.1. Protocolo SPI

El protocolo SPI usa paquetes de 32 bytes. Cada paquete contiene magic, version, comando, estado, secuencia, payload y CRC16.

Tabla 6.1: Estructura del paquete SPI

Campo	Tamano	Descripcion
Magic	2 bytes	0xA4, 0x29
Version	1 byte	Version de protocolo
Command	1 byte	Comando solicitado
Status	1 byte	Estado de respuesta
Flags	1 byte	Flags auxiliares
Sequence	2 bytes	Secuencia
Payload	22 bytes	Datos del comando
CRC16	2 bytes	Integridad del paquete

Comandos principales:

Tabla 6.2: Comandos SPI utilizados

Comando	Codigo	Funcion
PING	0x01	Verifica enlace SPI
GET_STATS	0x03	Lee contadores globales
GET_FILTER	0x04	Lee filtro activo
SET_FILTER	0x05	Configura filtro
RESET_STATS	0x06	Reinicia contadores
GET_LATEST_META	0x07	Lee metadata de snapshot
GET_LATEST_SLOT	0x08	Lee un slot de snapshot

6.2. SPI por PIO-frame

El slave SPI del sniffer se implementa en PIO. La state machine RX espera CS activo, lee 8 palabras de 32 bits y completa una trama de 32 bytes. La state machine TX entrega

la respuesta sincronizada con SCK. Esta implementacion evita depender del SPI slave hardware de la Pico y permite controlar el armado de paquetes completos.

6.3. DRDY

La linea DRDY conecta:

```
Pico GP20 -> Raspberry Pi GPIO25
```

Cuando el snapshot cambia, el sniffer sube DRDY. La 3B+ detecta el nivel y solicita metadata y slots por SPI. Si no se detecta DRDY, el bridge conserva un timeout periodico para mantener el estado actualizado.

6.4. Bridge HTTP/SPI

El bridge corre en la 3B+ y queda ligado a loopback:

```
http://127.0.0.1:5100
```

Sus responsabilidades son:

- inicializar SPI;
- manejar CS manual;
- leer DRDY;
- consultar snapshots;
- decodificar payloads;
- exponer `/stats`, `/metrics` y endpoints de control;
- separar errores de arranque de errores operativos.

Configuracion validada:

```
ARINC_SPI_HZ=8000000
ARINC_SPI_TRANSFER_MODE=pio-frame
ARINC_SPI_REQUEST_MODE=drdy
ARINC_SPI_MANUAL_CS=1
ARINC_SPI_CS_GPIO=8
ARINC_SPI_CS_SETUP_US=100
ARINC_SPI_CS_HOLD_US=100
ARINC_SPI_DRDY_GPIO=25
ARINC_SPI_DRDY_TIMEOUT_SEC=0.250
```

6.5. Dashboard Flask

Flask consulta al bridge local y publica una vista operativa en:

```
http://192.168.50.2:5000
```

La interfaz muestra:

- palabras aceptadas;
- paridad OK y errores de paridad;
- errores SPI;
- bitrate detectado;
- estado del supervisor;
- ultimas variables: temperatura, velocidad y altitud;
- distribucion por label y SSM.

6.6. Prometheus y Grafana

Flask expone `/metrics`. Prometheus scrapea esas metricas desde la notebook y Grafana permite visualizacion historica. La vista Flask se usa para operacion rapida; Prometheus/Grafana se usan para tendencia y diagnostico.

6.7. Supervisor

El supervisor corre como servicio systemd independiente. Consulta `/stats` y el estado de servicios. Clasifica el sistema en estados como `RUNNING`, `WAITING_SNIFFER`, `SPI_SYNCING`, `CABLE_FAULT`, `BRIDGE_FAULT` y `RECOVERING`.

Durante las corridas finales el supervisor permanecio en:

```
supervisor_state = RUNNING  
supervisor_health = ok
```

No ejecuto acciones correctivas durante las ocho horas de cada prueba final.

Capítulo 7

Metodologia de desarrollo y validacion

7.1. Procedimiento general

La metodologia combino desarrollo incremental, instrumentacion directa y corridas prolongadas. Cada cambio relevante se valido primero con diagnosticos cortos y luego con pruebas largas.

1. Compilacion de firmware TX, RX y SNIFFER.
2. Carga de UF2 en cada Pico.
3. Verificacion de conexion SPI con diagnostico 12/12.
4. Arranque de bridge, Flask y supervisor.
5. Verificacion de `/stats`.
6. Medicion instrumental con osciloscopio.
7. Registro prolongado con `stats_recorder.py`.
8. Analisis de `summary.json`, `events.csv` y PDF generado.

7.2. Actividades realizadas

Tabla 7.1: Actividades principales del trabajo

Actividad	Duracion relativa	Resultado
Organizacion del repositorio	Media	Componentes separados por funcion.
Firmware TX/RX logico	Alta	Palabras ARINC logicas a 100 kbps y 12.5 kbps.
Sniffer pasivo	Alta	Captura FWD/REV, filtro, paridad y snapshots.

Actividad	Duracion relativa	Resultado
Transporte SPI	Alta	Paquetes de 32 bytes en PIO-frame.
Bridge y dashboard	Media	HTTP/JSON, Flask y metricas.
Supervisor	Media	Vigilancia y recuperacion automatica.
Osciloscopio	Media	Validacion de bit time, RZ y diferencial logico.
Corridas largas	Alta	Evidencia de estabilidad de 8 h por velocidad.
Documentacion	Alta	README, procedimientos, evidencias y reportes.

7.3. Instrumental y medios

- placas Raspberry Pi Pico / Pico 2 W;
- Raspberry Pi 3B+;
- notebook con Prometheus y Grafana;
- osciloscopio Tektronix;
- conexion Ethernet directa;
- cables dupont y conexion de masa comun;
- firmware en C con Pico SDK;
- scripts Python para bridge, recorder y supervisor.

Capítulo 8

Resultados

8.1. Validacion con osciloscopio

Las capturas de osciloscopio confirman:

- amplitud logica individual aproximada de 3.3 V;
- retorno a cero en cada bit;
- bit time de 10 us a 100 kbps;
- lectura diferencial aproximada +3.3 V, 0 V y -3.3 V;
- palabras de 32 bits separadas por estado nulo.

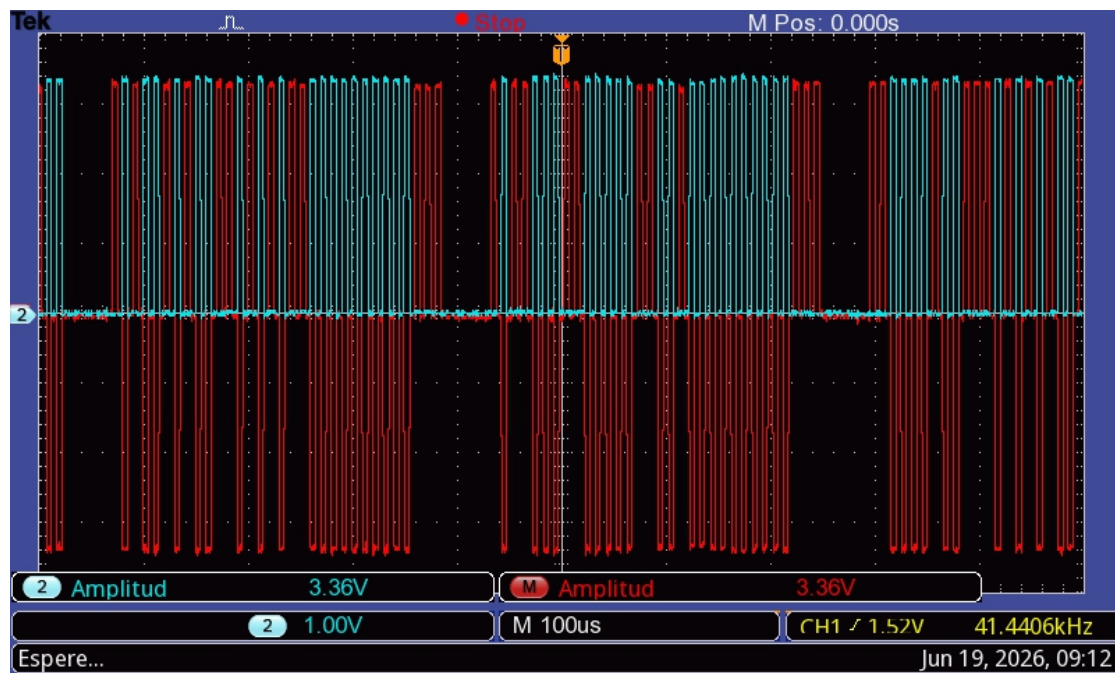


Figura 8.1: Captura MATH de senal diferencial logica. Se observan pulsos positivos, negativos y nivel nulo.

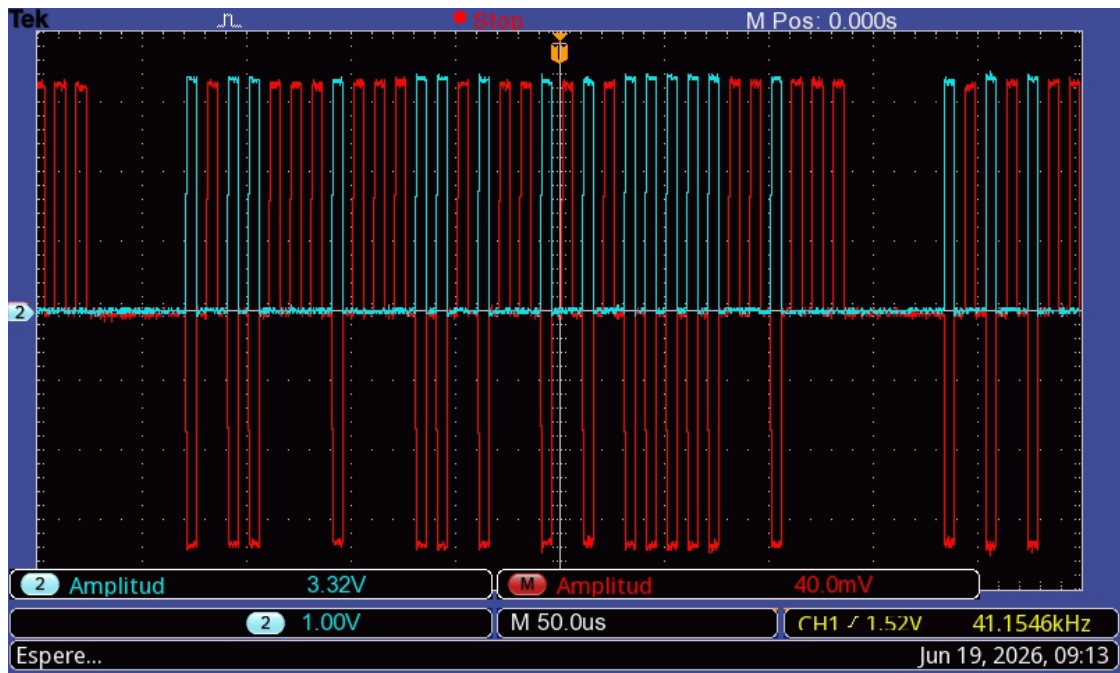
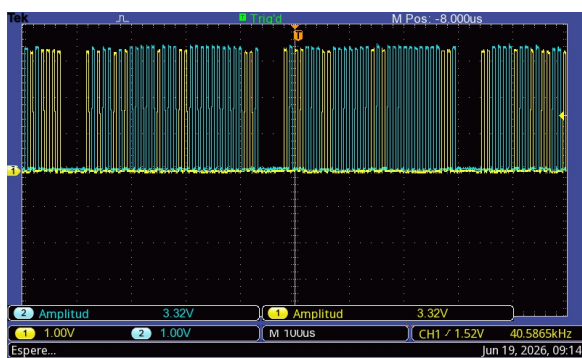
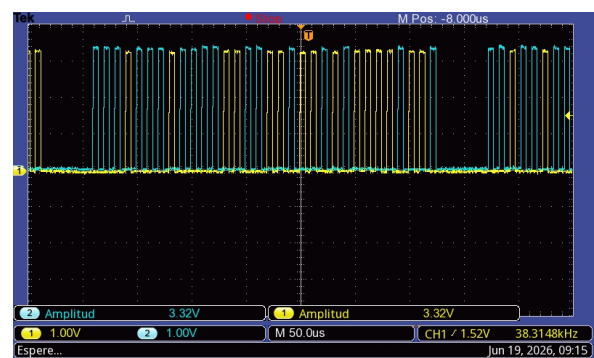


Figura 8.2: Secuencia de palabras sobre el canal observado con representacion diferencial.



(a) Canales individuales A/B.



(b) Tren de pulsos sin MATH.

Figura 8.3: Medicion de canales logicos individuales. Los canales no se pisan: representan polaridades opuestas o estado nulo.

8.2. Barrido de frecuencia SPI

El barrido solicitado incluyo frecuencias hasta 50 MHz. La conclusion practica fue seleccionar 8 MHz como perfil operativo con margen.

Tabla 8.1: Resumen del barrido SPI

Frecuencia	Resultado	Observacion
100 kHz a 8 MHz	PASS	Respuestas validas
9 MHz	PASS	Valido en prueba corta
10 MHz	Parcial	13/24 respuestas validas
16 MHz	FAIL	0/24
50 MHz	FAIL	Respuesta invalida / cero

La frecuencia de SPI no necesita coincidir con la velocidad ARINC. ARINC es el enlace observado; SPI es el transporte de telemetria ya decodificada entre sniffer y host.

8.3. Prueba de paridad estricta

Se uso una variante con inyeccion deliberada de errores de paridad para validar que el sniffer no entregue palabras corruptas a capas superiores.

Tabla 8.2: Prueba de paridad estricta

Metrica	Valor
Palabras recibidas	610905
Errores de paridad detectados	6109
Balance de clasificacion	0
Errores SPI operativos	0
Resultado	PASS

8.4. Corridas finales autorate

Las corridas finales validan las dos velocidades soportadas. Ambas duraron 8 h con una muestra cada 5 s.

Tabla 8.3: Comparacion de corridas finales

Metrica	12.5 kbps	100 kbps
Sesion	20260619_212806	20260620_065154
Duracion [s]	28801.451	28801.407
Muestras	5760	5760
Velocidad detectada [bps]	12500	100000
Palabras recibidas	9876638	76707606
Palabras aceptadas	2962992	23012283
Palabras filtradas	6913646	53695323
Tasa media aceptada [words/s]	102.876	798.999
Tasa maxima aceptada [words/s]	108.233	828.346
Errores de paridad	0	0
Errores SPI operativos	0	0
Overflow	0	0
Drops SPI	0	0
Resync FWD operativos	0	0
Muestras desconectadas	0	0
Veredicto	PASS	PASS

La relacion entre tasas medias aceptadas fue:

$$\frac{798,999}{102,876} = 7,77 \quad (8.1)$$

El valor es coherente con la relacion teorica 8:1 entre 100 kbps y 12.5 kbps. La diferencia corresponde a rafagas, pausas aleatorias del generador y muestreo discreto.

8.5. Balance de contadores

En ambas corridas se cumple:

$$\text{received_words} = \text{accepted_words} + \text{filtered_words} \quad (8.2)$$

Para 12.5 kbps:

$$9876638 = 2962992 + 6913646 \quad (8.3)$$

Para 100 kbps:

$$76707606 = 23012283 + 53695323 \quad (8.4)$$

Este cierre exacto muestra que la estadística conserva todo el tráfico clasificado: lo recibido queda aceptado o filtrado, sin una tercera categoría de pérdida.

8.6. Estado del supervisor

Durante las corridas finales:

- `supervisor_state = RUNNING;`

- `supervisor_health = ok;`
- `supervisor_issue = none;`
- acciones correctivas: 0;
- reinicios de servicio: 0;
- recuperaciones SPI: 0.

Capítulo 9

Costos, operacion e impacto

9.1. Inversion requerida

El prototipo se construyo con elementos de laboratorio y placas de desarrollo. La inversion se organiza por rubros.

Tabla 9.1: Rubros de inversion

Rubro	Elementos
Recursos humanos	Desarrollo de firmware, scripts, dashboard, pruebas y documentacion.
Equipamiento	Raspberry Pi Pico, Raspberry Pi 3B+, notebook, osciloscopio, cables y fuente.
Software	Pico SDK, Python, Flask, Prometheus, Grafana y herramientas de compilacion.
Infraestructura	Mesa de laboratorio, red Ethernet directa y alimentacion USB.
Insumos	Cables, conectores, almacenamiento y soporte de montaje.

El costo principal no estuvo en componentes electronicos, sino en horas de ingenieria, pruebas y documentacion. El uso de hardware disponible redujo la barrera de implementacion.

9.2. Costos de operacion y mantenimiento

Los costos de operacion son bajos:

- consumo electrico reducido de placas y notebook;
- mantenimiento de scripts y firmware;
- reemplazo de cables o conectores si fallan;
- conservacion de evidencias y backups del repositorio.

9.3. Viabilidad comercial

El trabajo se plantea como plataforma tecnica de laboratorio. Su valor esta en la capacidad de demostrar conceptos de comunicacion, captura pasiva y observabilidad. No se formula aqui una comercializacion de producto; por lo tanto el analisis economico se limita a la conveniencia tecnica y educativa del prototipo construido.

9.4. Impacto ambiental

El impacto ambiental es bajo. El sistema usa placas de bajo consumo y no requiere procesos contaminantes. La principal recomendacion operativa es reutilizar hardware disponible, conservar cables y gestionar correctamente los residuos electronicos al fin de su vida util.

9.5. Impacto social y academico

El aporte social y academico se concentra en:

- formacion practica en sistemas embebidos;
- instrumentacion de protocolos avionicos;
- trazabilidad de pruebas;
- integracion entre electronica, software y redes;
- disponibilidad de una base reproducible para evaluacion docente.

Capítulo 10

Dificultades y soluciones aplicadas

10.1. Errores SPI de arranque

Al iniciar bridge y placas en momentos distintos, el host podia intentar leer antes de que el sniffer estuviera listo. La solucion fue separar `spi_startup_errors` de `spi_errors`. Asi, los errores previos a la primera comunicacion valida no contaminan la estadistica operativa.

10.2. Transporte byte a byte

El transporte inicial byte a byte era funcional, pero no expresaba una trama atomica. Se implemento `pio-frame`, donde cada transaccion mueve un paquete completo de 32 bytes.

10.3. Polling

El polling continuo generaba consultas innecesarias. Se agrego `DRDY` para que el sniffer avise cuando hay snapshot nuevo. El sistema mantiene timeout como respaldo.

10.4. Reinicio o desconexion del sniffer

El reinicio del sniffer podia dejar al bridge sincronizado con una respuesta invalida. Se agrego reset de transporte por trama completa y recuperacion V5, con espera de reposo del canal antes de reactivar captura.

10.5. Gap temporal entre bits

Las mediciones iniciales mostraban separacion mayor a la esperada. El ajuste de PIO TX dejo la celda de bit en 10 us a 100 kbps, con 5 us activos y retorno a nulo, validado por osciloscopio.

10.6. Robustez de servicios

Bridge y Flask se instalaron como servicios `systemd` con autoarranque. El supervisor verifica estado y expone su condicion por `/stats` y `/metrics`.

Capítulo 11

Conclusiones

El trabajo alcanzo el objetivo propuesto: se implemento y valido una plataforma de laboratorio capaz de generar, recibir, sniffear y visualizar trafico ARINC 429 logico. La solucion integra hardware embebido, PIO, SPI, HTTP, dashboard, metricas y registro estadistico.

Los resultados finales son consistentes y verificables. Las corridas de ocho horas a 100 kbps y 12.5 kbps cerraron con cero errores SPI operativos, cero errores de paridad, cero overflow, cero drops SPI, cero resync FWD operativos y supervisor en estado sano. La tasa observada escala aproximadamente 8:1 entre ambas velocidades, como predice la relacion teorica.

La medicion con osciloscopio confirmo el retorno a cero, la duracion de bit y la forma diferencial logica. El transporte SPI por tramas completas, la linea DRDY y el supervisor convierten al sistema en una plataforma operable y no solo en una prueba aislada de firmware.

La arquitectura final queda ordenada por responsabilidades: TX genera trafico, RX reconstruye palabras, SNIFFER observa pasivamente, la 3B+ traduce a HTTP y la notebook registra y visualiza. Esta separacion facilita explicar el sistema, repetir ensayos y auditar resultados.

Capítulo 12

Referencias y bibliografía

Bibliografía

- [1] ARINC Specification 429. *Mark 33 Digital Information Transfer System*. Aeronautical Radio, Inc.
- [2] Raspberry Pi Ltd. *RP2040 Datasheet*. Programmable I/O, GPIO, DMA y perifericos.
- [3] Raspberry Pi Ltd. *Raspberry Pi Pico C/C++ SDK*.
- [4] Pallets Projects. *Flask Documentation*.
- [5] Prometheus Authors. *Prometheus Documentation*.
- [6] Grafana Labs. *Grafana Documentation*.
- [7] Repositorio local del proyecto PAMPA / ARINC 429. *ARINC_SPI*, rama **ARINC**, tag **arinc-logic-v1.1.0**.

Apéndice A

Comandos operativos principales

A.1. Servicios en Raspberry Pi 3B+

```
systemctl status arinc-sniffer-bridge arinc-dashboard-flask arinc-supervisor --no-pager
curl http://127.0.0.1:5100/stats
curl http://127.0.0.1:5100/supervisor/status
```

A.2. Prometheus y Grafana en notebook

```
cd "C:\Users\usuario\RASPI PICO\PAMPA\ARINC_SPI\prometheus"
powershell -ExecutionPolicy Bypass -File .\start_prometheus_grafana.ps1
```

A.3. Registro estadístico

```
cd ~/PAMPA/HOST_3b+
source .venv/bin/activate
nohup python3 stats_recorder.py start \
  --name corrida_final_noche_autorate \
  --duration 28800 \
  --interval 5 \
  --tx-firmware arinc_tx_arinc429_logic_stream_autorate \
  --rx-firmware arinc_rx_arinc429_logic_stream \
  --sniffer-firmware sniffer_arinc429_logic_pio_frame \
  > stats_recorder.log 2>&1 &
```

Apéndice B

Archivos de evidencia

- docs/evidencia_corridas_finales_autorate_8h.md
- docs/evidencia_autorate_100k_12k5.md
- docs/evidencia_osciloscopio_arinc429_20260619.md
- docs/evidencia_barrido_frecuencia_spi.md
- docs/evidencia_recuperacion_fallas_v5.md
- docs/entrega_tutor/datos/summary_corrida8h_100k.json
- docs/entrega_tutor/datos/summary_corrida8h_12k5.json
- docs/entrega_tutor/reportes/reporte_corrida8h_100k.pdf
- docs/entrega_tutor/reportes/reporte_corrida8h_12k5.pdf